PRACTICAL LAB REPORT: FILE INTEGRITY & SHA-256 HASHING

DAY 34 PRACTICAL LAB - FILE INTEGRITY REPORT

Student Name: Jiwosh Shrestha	**Platform:** Local Defensive Integrity Lab
Lab Focus: OWASP A08 - Software / Data Integrity	**Assignment:** Day 34 - File Integrity & SHA-256 Hashing

PART 1: PRACTICAL LAB EXECUTION STEPS

TASK 1: Create Local Test File

Created a local test file using the authorized bash command:

```
$ echo "Jayvik Labs Day 34 Integrity Lab" > day34_original.txt
$ cat day34_original.txt
Jayvik Labs Day 34 Integrity Lab
```

TASK 2: Calculate Original SHA-256

Computed the cryptographic SHA-256 digest of the unmodified test file:

```
$ sha256sum day34_original.txt
bc04e4ce99455d1aa7677be5ce79f07e20037f43d70fa05db77416115fa16a30  day34_original.txt
```

TASK 3: Make Authorized Modification

Appended a single authorized line to the file to modify its contents:

```
$ echo "Modified" >> day34_original.txt
$ cat day34_original.txt
Jayvik Labs Day 34 Integrity Lab
Modified
```

TASK 4 & 5: Recalculate Hash & Compare Results

Recalculated the SHA-256 hash after making the modification and compared the output digests:

```
$ sha256sum day34_original.txt
46dbe19421721a17042be15f53b4fb7157b17ebad9435dc687a9375e996e84f3  day34_original.txt
```

Action / State	SHA-256 Hash Digest
Original Hash	`bc04e4ce99455d1aa7677be5ce79f07e20037f43d70fa05db77416115fa16a30`
Modified Hash	`46dbe19421721a17042be15f53b4fb7157b17ebad9435dc687a9375e996e84f3`

Comparison Outcome: The original and modified hashes are completely different, demonstrating that even a minimal modification renders an entirely distinct hash digest.

PART 2: INTEGRITY ANALYSIS & DOWNLOAD VERIFICATION

TASK 6: Why Did the Hash Change?

SHA-256 is a cryptographic hash function. Even a single-bit or single-byte change in the input file produces a completely different, unpredictable output—a fundamental property known as the avalanche effect. Appending the word "Modified" altered the underlying binary content of the file, resulting in an entirely distinct digest. This property makes cryptographic hashes ideal for detecting unauthorized alterations.

TASK 7: How Can a Software Publisher Use Hashes for Download Verification?

A software publisher computes the SHA-256 (or stronger) hash of a software installer or package and publishes it on a trusted channel (e.g., official website, signed release bulletin, or package manager metadata).

After downloading the software package, the end user recomputes the hash locally on their machine using command-line tools (such as sha256sum or CertUtil) and compares it with the published hash. If the hashes match identically, the user is assured that the file was not altered or corrupted during download (e.g., by a Man-in-the-Middle attack or a compromised mirror).

TASK 8: Why Must the Expected Hash Itself Come from a Trusted Source?

A hash function only proves that two pieces of data are identical; it does not independently verify origin or authenticity. If an attacker replaces a software package with a malicious version on an untrusted mirror, they can also easily compute and publish the hash of that malicious payload on the same compromised site.

If the user checks the file against an attacker-supplied hash, the hash verification will succeed despite the software being malicious. Therefore, the expected hash must be retrieved from a trusted, authenticated channel—ideally one protected by digital signatures, HTTPS with certificate pinning, or public key infrastructure (PKI).

Digital signatures are stronger than plain hashes because they combine integrity (via cryptographic hashing) with authenticity (via private-key signing). A digital signature proves both that the file remains unmodified and that it was produced by the legitimate private key holder.